



SUNSPOT

Trova, condividi e vivi i tramonti e le albe più belli

Programmazione di Dispositivi Mobili

A.A. 2025-2026

Realizzato da:

Valentina Felici 916171

Indice

Introduzione	3
SunSpot	3
Scelta del Progetto & API	3
Organizzazione	3
Design	4
Temi	4
Material components	4
Funzionalità & Screenshot	5
Autenticazione: Login e Registrazione	5
Home: meteo attuale e spot nelle vicinanze	5
Mappa: visualizzazione ed esplorazione degli spot	6
Aggiungere un nuovo spot	7
Visualizzare i dettagli di uno Spot	8
Profilo utente	9
Architettura	11
Database	11
Model	12
Repository	12
Service	12
Source	12
UI	13
ViewModel	13
Utilities	13
Librerie utilizzate	15
Room	15
Retrofit e OkHttp	15
Gson	15
Firebase	15
osmdroid	15
Glide	15
Play Services Location	15
AndroidX ExifInterface	15
Sviluppi futuri & Idee	16

Introduzione

SunSpot

Perché SunSpot?

SunSpot è un'applicazione mobile pensata per chi ama i tramonti e le albe. Permette agli utenti di scoprire, sulla mappa, i punti panoramici migliori per ammirare questi fenomeni, di aggiungerne di nuovi condividendoli con la community, e di consultare in tempo reale le condizioni meteo e gli orari precisi di alba e tramonto per ciascuno spot.

L'applicazione si riferisce a questi luoghi come "Spot", ciascuno classificato come punto di osservazione per l'alba o per il tramonto.

SunSpot nasce dall'idea di unire due esigenze: da un lato la voglia di scoprire nuovi luoghi suggestivi vicino a sé o durante un viaggio, dall'altro la necessità pratica di sapere in anticipo se le condizioni meteo permetteranno di godersi lo spettacolo. Per questo l'app integra, per ogni spot, una previsione meteo aggiornata in tempo reale.

Un'ulteriore funzionalità centrale di SunSpot è la dimensione sociale e collaborativa: ogni utente registrato può aggiungere un nuovo spot, corredato di nome, descrizione, posizione e fotografia, rendendolo immediatamente visibile a tutta la community. Ogni utente può inoltre salvare i propri spot preferiti e consultare in ogni momento l'elenco degli spot che ha personalmente contribuito ad aggiungere.

Scelta del Progetto & API

Il punto di partenza del progetto è stata la volontà di costruire un'applicazione con una forte componente geografica e social, che unisse dati in tempo reale (il meteo) a contenuti generati dagli utenti (gli spot). Mi sono chiesta come rendere più semplice, per chi ama la fotografia paesaggistica o semplicemente vuole godersi un tramonto, scoprire il posto giusto al momento giusto.

Per il meteo ho scelto Open-Meteo, un'API gratuita e open source che non richiede alcuna chiave di autenticazione né limiti di utilizzo per uso non commerciale, e che fornisce dati meteo attuali oltre agli orari precisi di alba e tramonto per qualunque coordinata geografica del mondo.

Per la gestione dei dati condivisi tra utenti (gli spot) e per l'autenticazione ho scelto la piattaforma Firebase di Google, che offre un ecosistema completo di servizi cloud: Firebase Authentication per la gestione di login e registrazione, Cloud Firestore come database condiviso per gli spot, e Firebase Realtime Database per i profili utente.

Per la componente cartografica ho scelto osmdroid, una libreria open source basata sui dati di OpenStreetMap, che mi ha permesso di integrare una mappa interattiva completa (zoom, pan, marker personalizzati) senza dover configurare fatturazione o chiavi API a pagamento.

Organizzazione

Prima di iniziare a lavorare al progetto, ho pianificato il lavoro e organizzato il flusso di sviluppo utilizzando diversi strumenti a mio favore:

GitHub: un servizio di hosting per progetti software che mi ha permesso di avere un controllo di versione sempre aggiornato durante tutto lo sviluppo dell'applicazione.

Android Studio: l'IDE ufficiale per lo sviluppo Android, utilizzato per scrivere, testare e debuggare l'intera applicazione.

Firebase Console: la piattaforma web per configurare e monitorare i servizi Firebase utilizzati dall'app (Authentication, Firestore, Realtime Database).

Documentazione Android e Open-Meteo: le documentazioni ufficiali che mi hanno guidata nello sviluppo dell'applicazione, della sua architettura e nell'integrazione dell'API meteo.

ChatGPT (OpenAI): utilizzato per la generazione del logo dell'applicazione.

Design

Temi

Invece di creare temi personalizzati manualmente, mi sono affidata ai temi dinamici Material 3 integrati in Android. Il colore dinamico consente agli utenti di personalizzare i propri dispositivi in modo che siano in linea con la combinazione di colori dello sfondo personale o di un colore selezionato nel selettore di sfondi.

L'interfaccia di SunSpot è stata progettata secondo un principio di minimalismo: le schermate utilizzano prevalentemente toni neutri della palette Material 3 (superficie, superficie container), riservando l'uso di colore a pochi elementi mirati, come le icone tematiche di alba e tramonto, in modo da mantenere l'attenzione dell'utente sui contenuti (le fotografie degli spot) piuttosto che su una veste grafica invadente.

Material components

I Material Component di Google sono blocchi da costruzione interattivi per la creazione di interfacce utente. Nello sviluppo di SunSpot ho utilizzato diversi componenti, elencati di seguito insieme al loro ruolo nell'applicazione.

osmdroid MapView

Permette di costruire mappe dinamiche e interattive basate su OpenStreetMap, di posizionare marker personalizzati (con un'icona a goccia in stile Material coerente con il tema dell'app) e di intercettare i tocchi dell'utente sulla mappa per l'aggiunta di nuovi spot.

Bottom Navigation

Permette agli utenti di muoversi tra le tre sezioni principali dell'app: Home, Mappa e Profilo utente.

Material Card View

Utilizzate estensivamente per la visualizzazione degli spot (nella lista degli spot vicini, nei “miei spot”, nei preferiti), per la card del meteo attuale e per le card di alba e tramonto nella schermata Home.

Toggle Button Group

Utilizzato nella schermata di aggiunta di un nuovo spot per permettere all'utente di scegliere in modo rapido e visivo se lo spot è dedicato all'osservazione dell'alba o del tramonto.

Snackbar

Mostra brevi messaggi di errore o conferma nella parte inferiore dello schermo, ad esempio in caso di credenziali errate durante il login.

TextInputLayout

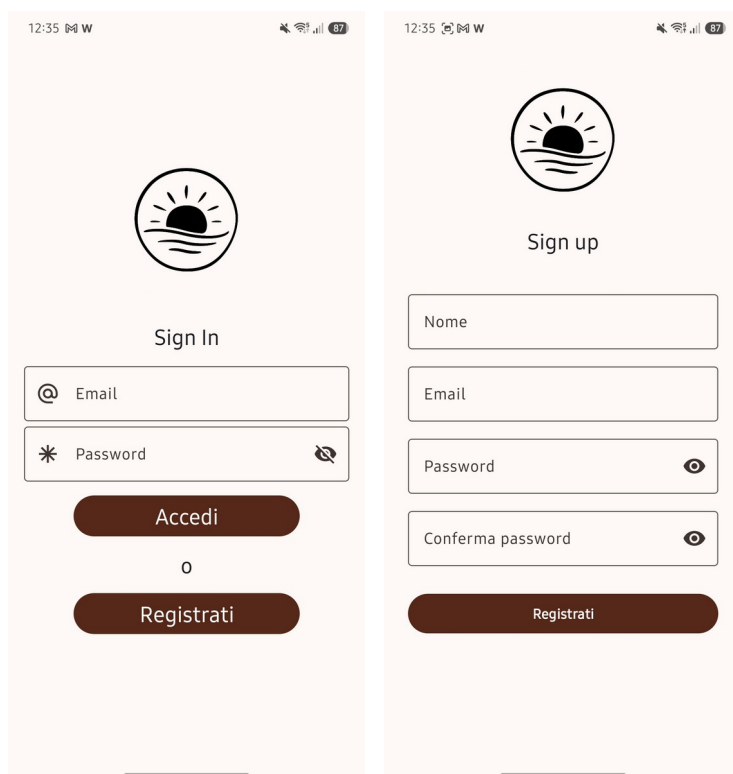
Utilizzato in tutti i form dell'applicazione (login, registrazione, aggiunta spot) per la validazione e la visualizzazione di eventuali errori nei campi di testo.

Funzionalità & Screenshot

Seguono tutte le funzionalità di SunSpot con i relativi screenshot.

Autenticazione: Login e Registrazione

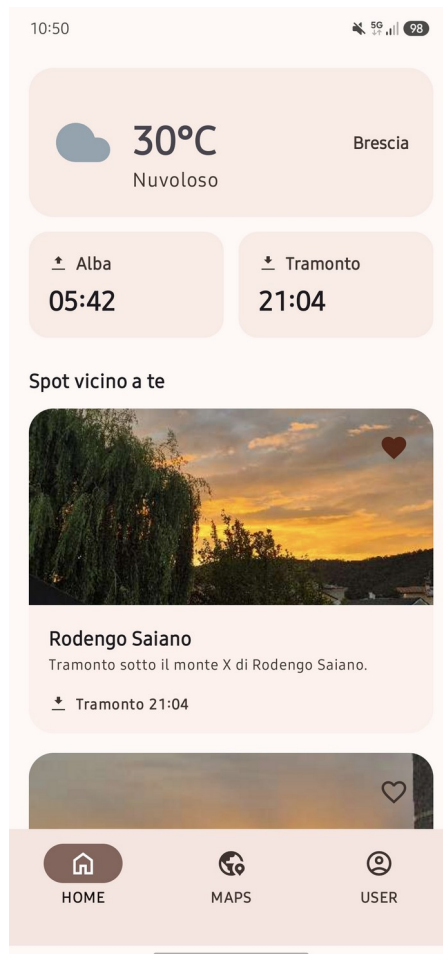
All'avvio, l'applicazione mostra la schermata di accesso, dalla quale l'utente può effettuare il login con email e password oppure registrarsi per creare un nuovo account. La registrazione richiede nome, email e password (con conferma), e al termine crea l'utente sia su Firebase Authentication sia, con i relativi dati di profilo, su Firebase Realtime Database. Se l'utente ha già una sessione attiva, l'app salta automaticamente la schermata di login e apre direttamente la Home.



Schermate di accesso e registrazione

Home: meteo attuale e spot nelle vicinanze

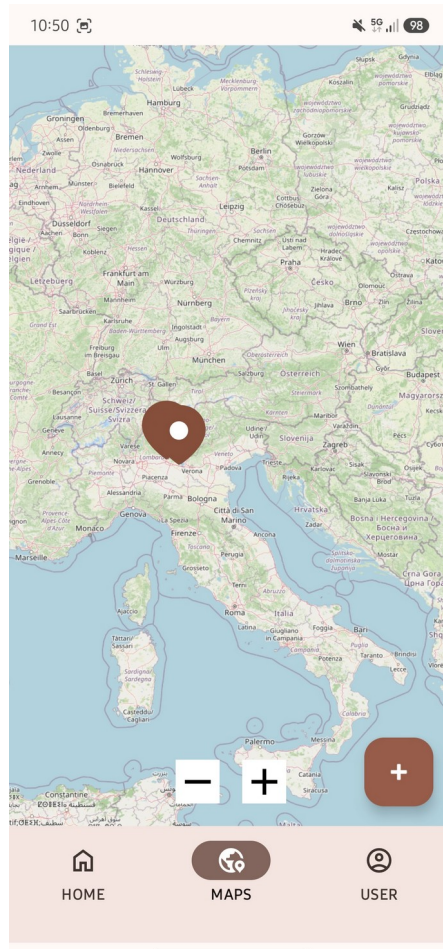
Una volta effettuato l'accesso, la schermata Home richiede il permesso di localizzazione e, una volta ottenuta la posizione dell'utente, mostra il nome della località in cui si trova, la temperatura e le condizioni meteo attuali (con un'icona dedicata a seconda che sia sereno, nuvoloso, piovoso, nevoso, con nebbia o temporale), e gli orari di alba e tramonto della giornata, ciascuno in una card dedicata. Sotto, una lista verticale mostra gli spot presenti entro un raggio di 50 km dalla posizione dell'utente, ordinati per distanza crescente, ciascuno con l'indicazione se si tratta di uno spot per l'alba o per il tramonto e il relativo orario.



Schermata Home: meteo attuale, alba/tramonto e spot vicini

Mappa: visualizzazione ed esplorazione degli spot

La schermata Mappa mostra tutti gli spot condivisi dagli utenti sotto forma di marker geolocalizzati. Toccando un marker si accede alla schermata di dettaglio dello spot corrispondente. Un pulsante flottante permette di attivare la modalità di aggiunta di un nuovo spot: una volta attivata, toccando un punto sulla mappa si viene indirizzati al form di creazione di un nuovo spot, con le coordinate già precompilate in base al punto selezionato.



Schermata Mappa: marker degli spot e pulsante di aggiunta

Aggiungere un nuovo spot

Dal form di aggiunta, l'utente può inserire il nome dello spot, una breve descrizione, una fotografia scelta dalla galleria del dispositivo e selezionare, tramite un gruppo di pulsanti a scelta singola, se lo spot è dedicato all'osservazione dell'alba o del tramonto. Una volta salvato, lo spot viene caricato su Cloud Firestore e diventa immediatamente visibile a tutti gli utenti dell'applicazione.

10:55

59

97

Aggiungi spot



Nome dello spot

Jardim do Morro, Porto

Descrizione

Tramonto sulla città di Porto, punto panoramico facilmente raggiungibile a piedi, passando per il ponte Dom Luís I.
Spot molto battuto dai turisti.

Tipo di spot

Alba

Tramonto

Salva spot

HOME

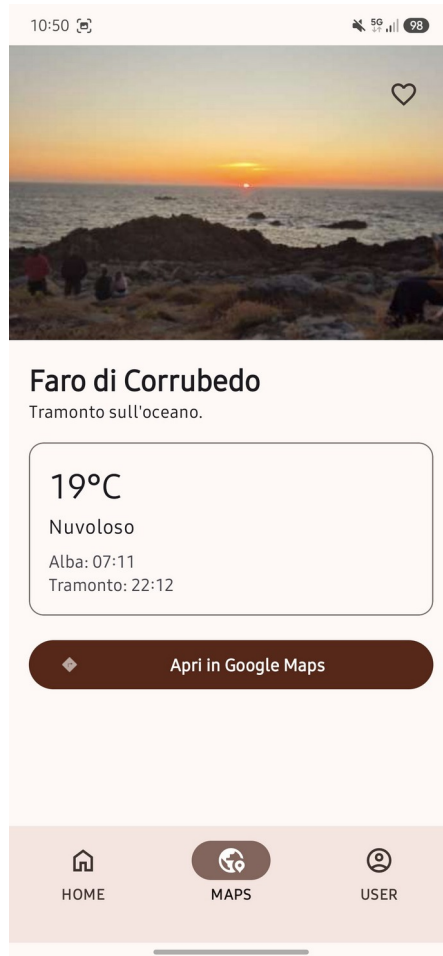
MAPS

USER

Schermata di aggiunta di un nuovo spot

Visualizzare i dettagli di uno Spot

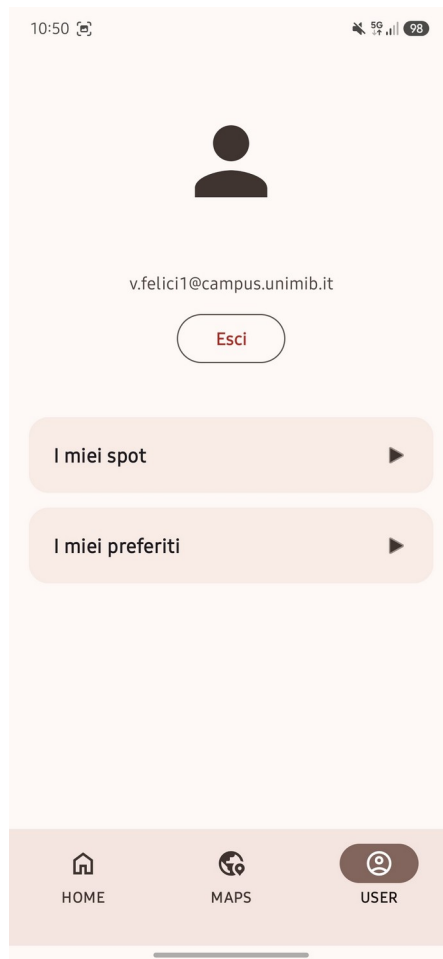
Toccando uno spot, sia dalla mappa sia da una qualsiasi lista, si apre una schermata di dettaglio a schermo intero che mostra la fotografia dello spot, il nome, la descrizione, le condizioni meteo attuali richieste in tempo reale per quella specifica posizione geografica, gli orari di alba e tramonto, un pulsante per aprire la posizione direttamente in Google Maps e un pulsante a forma di cuore per aggiungere o rimuovere lo spot dai propri preferiti. Se lo spot è stato aggiunto dall'utente attualmente connesso, viene mostrato anche un pulsante per eliminarlo definitivamente.



Schermata di dettaglio di uno spot

Profilo utente

La schermata Utente mostra l'avatar, il nome e l'email dell'utente connesso, insieme a due sezioni di accesso rapido: "I miei spot", che elenca tutti gli spot aggiunti dall'utente, e "Preferiti", che elenca tutti gli spot che l'utente ha contrassegnato con un cuore. Da questa schermata è inoltre possibile effettuare il logout, che termina la sessione e riporta l'utente alla schermata di accesso.



Schermata del profilo utente

Architettura

SunSpot contiene più componenti, tra cui Activity, Fragment, Service, Repository, ViewModel e Utility.

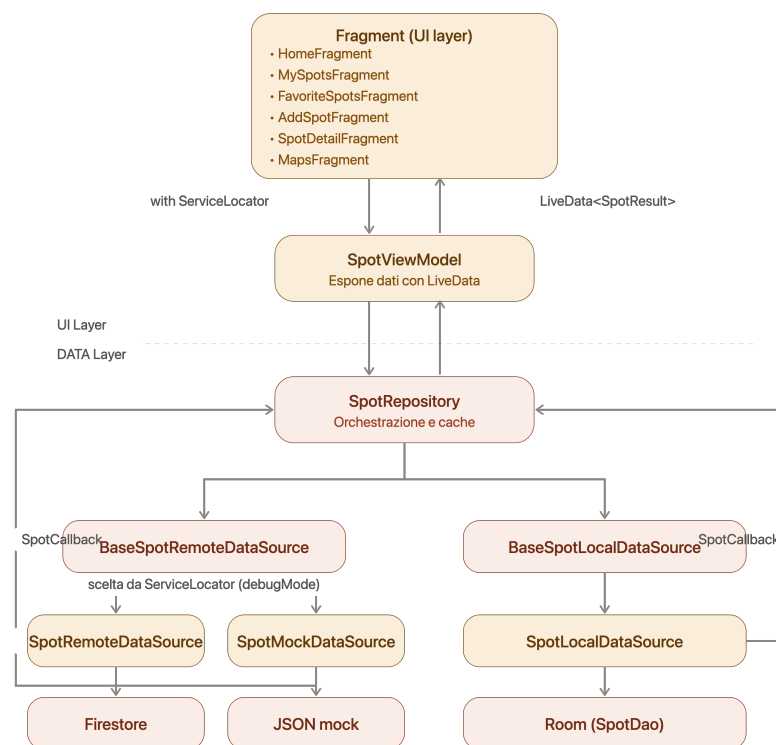
L'architettura definisce i confini tra le parti dell'app e le responsabilità che ciascuna parte dovrebbe avere. Il principio più importante seguito è quello della separazione delle responsabilità: le Activity e i Fragment contengono solo la logica che gestisce le interazioni dell'interfaccia utente, mentre la logica di business e l'accesso ai dati sono demandati ai livelli sottostanti.

Un altro principio seguito è l'utilizzo di modelli di dati indipendenti dagli elementi dell'interfaccia utente e dagli altri componenti dell'app, resi persistenti tramite database locale (Room) e cloud (Firestore).

Il ruolo del livello UI è visualizzare i dati dell'applicazione sullo schermo. Ogni volta che i dati cambiano, a causa dell'interazione dell'utente o di un input esterno (ad esempio una risposta di rete), l'interfaccia utente si aggiorna automaticamente per riflettere le modifiche.

SunSpot adotta il pattern architetturale Model-View-ViewModel (MVVM): i Fragment e le Activity sono i componenti dello strato UI, mentre il ViewModel fa da tramite tra la View e il Model. Grazie a MutableLiveData e agli Observer, i Fragment sono in grado di rispondere automaticamente ai cambiamenti dei dati.

Schema architetturale generale di SunSpot



Database

Il package database comprende SpotDao, un'interfaccia che definisce le query eseguibili sulla tabella Spot del database Room, e SpotRoomDatabase, la classe astratta singleton per accedere all'istanza del database, con un ExecutorService dedicato per l'esecuzione delle operazioni fuori dal thread principale.

Room viene utilizzato come cache locale degli spot: ad ogni sincronizzazione con Firestore, i dati locali vengono sostituiti integralmente con quelli più recenti provenienti dal cloud, garantendo che Room rifletta sempre fedelmente lo stato remoto, senza necessità di complesse logiche di merge.

Model

Le varie strutture all'interno del Model rappresentano i tipi di dati utilizzati da SunSpot, indipendenti dall'interfaccia utente:

Spot: rappresenta un punto panoramico, con nome, descrizione, coordinate geografiche, fotografia, tipo (alba o tramonto), identificativo dell'utente che lo ha creato e l'elenco degli utenti che lo hanno aggiunto ai preferiti. Implementa sia l'annotazione `@Entity` di Room sia l'interfaccia `Parcelable`, per essere sia persistito localmente sia trasferito tra Fragment tramite Bundle.

User: rappresenta il profilo di un utente registrato (identificativo univoco, nome, email).

WeatherResponse: rappresenta la risposta dell'API Open-Meteo, con le sotto-strutture `CurrentWeather`, `DailyWeather`, `CurrentUnits` e `DailyUnits`.

SpotResult, WeatherResult, UserResult: classi che rappresentano l'esito di un'operazione asincrona, distinguendo tra un risultato di successo (con i dati richiesti) e un risultato di errore (con il relativo messaggio).

Repository

Il livello dati è costituito da repository che possono contenere una o più origini dati. È stata creata una classe repository per ogni tipo di dato gestito nell'app: `SpotRepository`, `WeatherRepository` e `UserRepository`.

Le classi Repository sono responsabili di:

- Esporre i dati al resto dell'app tramite oggetti `MutableLiveData` osservabili.
- Centralizzare le modifiche ai dati.
- Coordinare le diverse origini dati (locale e remota).
- Astrarre le origini dati dal resto dell'app.

Ogni Repository lavora con una o più origini dati: `SpotRepository` coordina Cloud Firestore (origine remota) e Room (cache locale); `WeatherRepository` lavora esclusivamente con l'API Open-Meteo tramite Retrofit, poiché il dato meteo non necessita di persistenza locale; `UserRepository` coordina Firebase Authentication (per il login e la registrazione) e Firebase Realtime Database (per i dati di profilo).

Service

Contiene l'interfaccia `WeatherAPIService` per l'interrogazione dell'API Open-Meteo tramite Retrofit, con i parametri di latitudine, longitudine, dati richiesti (meteo attuale e giornaliero) e fuso orario.

Source

Sono le origini dei dati di SunSpot. Per ogni tipo di dato gestito dall'applicazione sono state definite classi base astratte (`BaseSpotRemoteDataSource`, `BaseSpotLocalDataSource`, `BaseUserAuthenticationRemoteDataSource`, `BaseUserDataRemoteDataSource`) e le relative implementazioni concrete:

SpotRemoteDataSource: legge e scrive gli spot su Cloud Firestore.

SpotMockDataSource: implementazione alternativa che legge dati di prova da un file JSON incluso negli asset dell'app, utile in fase di sviluppo per testare l'interfaccia senza dipendere da una connessione di rete.

SpotLocalDataSource: legge e scrive gli spot sul database Room locale.

WeatherRemoteDataSource: effettua le chiamate all'API Open-Meteo tramite Retrofit.

UserAuthenticationFirebaseDataSource: gestisce login, registrazione e logout tramite Firebase Authentication.

UserFirebaseDataSource: legge e scrive i dati di profilo dell'utente su Firebase Realtime Database.

La scelta tra la sorgente reale (Firestore) e quella di prova (Mock) avviene tramite un flag booleano di configurazione (`debug_mode`), gestito centralmente dal `ServiceLocator`.

UI

Tutto ciò che comprende lo strato UI, che si interfaccia e interagisce con l'utente. Ne fanno parte le Activity e i Fragment.

Activity

WelcomeActivity: Activity di autenticazione che ospita i Fragment di Login e Registrazione.

HomeActivity: Activity principale dell'app, che ospita la Bottom Navigation e i Fragment relativi a Home, Mappa e Profilo utente.

Fragment

LoginFragment: Fragment per l'accesso a SunSpot tramite email e password.

SignUpFragment: Fragment per la registrazione di un nuovo account, con conferma della password.

HomeFragment: mostra il meteo attuale, la località, l'alba e il tramonto della giornata, e la lista degli spot nelle vicinanze.

MapsFragment: ospita la mappa interattiva e i marker degli spot, con la funzionalità di aggiunta di un nuovo spot.

AddSpotFragment: Fragment per la creazione di un nuovo spot, con selezione della fotografia e del tipo (alba/tramonto).

SpotDetailFragment: mostra il dettaglio di uno spot selezionato, con meteo in tempo reale, preferiti ed eliminazione.

UserFragment: mostra il profilo dell'utente connesso e l'accesso rapido a "I miei spot" e "Preferiti".

MySpotsFragment: elenca tutti gli spot aggiunti dall'utente connesso.

FavoriteSpotsFragment: elenca tutti gli spot che l'utente connesso ha contrassegnato come preferiti.

ViewModel

Le classi ViewModel sono progettate per archiviare e gestire i dati relativi all'interfaccia utente in modo indipendente dal ciclo di vita di Activity e Fragment. Sopravvivono ai cambi di configurazione (ad esempio la rotazione dello schermo), evitando così di dover ripetere inutilmente le richieste di rete già effettuate.

SpotViewModel: ViewModel per la gestione degli spot (lettura, aggiunta, aggiornamento dei preferiti, eliminazione).

WeatherViewModel: ViewModel per le richieste meteo verso Open-Meteo.

UserViewModel: ViewModel per l'autenticazione e la gestione della sessione utente.

Ogni ViewModel viene istanziato tramite una ViewModelFactory dedicata (SpotViewModelFactory, WeatherViewModelFactory, UserViewModelFactory), necessaria poiché i ViewModel dell'applicazione richiedono un costruttore personalizzato con il relativo Repository.

Utilities

Contiene diversi elementi di utilità per facilitare le operazioni di sviluppo ed evitare la ripetizione di codice all'interno del progetto.

Constants: variabili costanti usate globalmente nel progetto (endpoint delle API, chiavi dei Bundle, messaggi di errore, timeout di cache).

Converters: TypeConverter di Room necessari per la persistenza di campi non primitivi, come le liste di stringhe (fotografie e utenti che hanno messo un preferito).

LocationUtil: gestisce il recupero della posizione GPS dell'utente, il calcolo della distanza tra due coordinate e il reverse geocoding per ottenere il nome della località.

WeatherUtil: traduce i codici meteo standard WMO restituiti da Open-Meteo in descrizioni testuali e icone, e formatta gli orari di alba e tramonto.

ImageUtil: comprime, ridimensiona e corregge l'orientamento (tramite i metadati EXIF) delle fotografie scelte dall'utente, prima di codificarle in Base64 per il salvataggio su Firestore.

NetworkUtil: verifica la disponibilità di una connessione di rete attiva.

ServiceLocator: memorizza e fornisce le istanze dei servizi (Retrofit, Room, Firebase) e assembla i Repository con le rispettive DataSource, decidendo centralmente se utilizzare le sorgenti dati reali o quelle di prova.

SharedPreferencesUtils: classe di utilità per la gestione delle preferenze semplici dell'utente durante il runtime.

JSONParserUtils: classe generica per la lettura e il parsing di file JSON di prova dagli asset dell'applicazione.

Librerie utilizzate

Seguono le librerie utilizzate nello sviluppo dell'applicazione SunSpot.

Room

Le app che gestiscono quantità non trascurabili di dati strutturati possono trarre grandi vantaggi dalla memorizzazione locale di questi dati. La libreria di persistenza Room fornisce un livello di astrazione su SQLite per consentire un accesso fluido al database, sfruttando al contempo tutta la potenza di SQLite. In SunSpot, Room viene utilizzato come cache locale degli spot condivisi.

Retrofit e OkHttp

Retrofit è una libreria per Android e Java che semplifica la comunicazione con API REST, trasformando le risposte JSON direttamente in oggetti Java. OkHttp è il client HTTP sottostante, ottimizzato per la velocità e la gestione delle connessioni. In SunSpot, Retrofit viene utilizzato per interrogare l'API Open-Meteo.

Gson

Libreria utilizzata per convertire automaticamente le risposte JSON dell'API Open-Meteo negli oggetti Java del Model dell'applicazione.

Firebase

Piattaforma Backend-as-a-Service (BaaS) utilizzata per tre servizi distinti: Firebase Authentication per la registrazione e il login degli utenti tramite email e password, Cloud Firestore come database condiviso e in tempo reale per gli spot creati dagli utenti, e Firebase Realtime Database per la gestione dei profili utente.

osmdroid

Libreria open source per l'integrazione di mappe interattive basate su OpenStreetMap, utilizzata per la visualizzazione degli spot sotto forma di marker geolocalizzati e per l'interazione (zoom, spostamento, tocco) con la mappa.

Glide

Libreria per il caricamento efficiente di immagini da URL remoti, con gestione automatica della cache e di un'immagine segnaposto durante il caricamento.

Play Services Location

Componente di Google Play Services utilizzato per ottenere in modo efficiente la posizione geografica dell'utente tramite FusedLocationProviderClient.

AndroidX ExifInterface

Libreria utilizzata per leggere i metadati di orientamento (EXIF) delle fotografie selezionate dalla galleria del dispositivo, in modo da correggerne automaticamente la rotazione prima del salvataggio.

Sviluppi futuri & Idee

Essendo SunSpot sviluppato in maniera modulare, seguendo il pattern Repository, sarebbe possibile in futuro sostituire o affiancare nuove origini dati (ad esempio un'API meteo diversa) senza dover modificare il resto dell'applicazione.

- Aggiungere la possibilità di caricare più fotografie per ogni spot, mostrandole in un carosello nella schermata di dettaglio.
- Introdurre un sistema di recensioni e commenti sugli spot, per raccogliere le impressioni degli utenti che li hanno visitati.
- Permettere all'utente di personalizzare il raggio di ricerca degli "spot vicini" nella schermata Home, attualmente fisso a 50 km.
- Implementare un sistema di notifiche push che avvisi l'utente quando le condizioni meteo di uno spot preferito sono ideali per l'osservazione del tramonto o dell'alba.
- Introdurre una modalità di ricerca testuale degli spot, oltre alla sola esplorazione tramite mappa e posizione.
- Creare una vera e propria community tra gli utenti, introducendo un sistema di badge e riconoscimenti da sbloccare, ad esempio per il numero di spot visitati o per il numero di spot aggiunti alla piattaforma, incentivando così la partecipazione attiva e la scoperta di nuovi luoghi.
- Realizzare una sequenza di tutorial guidato per mostrare all'utente le funzionalità dell'app al primo avvio.